

SOUTENANCE D'ARCHITECTURE · PROJET 13

D'un POC RAG à un MVP scalable

Agent conversationnel culturel — Pays de la Loire

RAG hybride

Mémoire

Géo

Web temps réel

Monitoring

L'existant — le POC P11

Ce qui fonctionne déjà

- **RAG fonctionnel**
LangChain + Mistral + FAISS
- **Retriever hybride**
BM25 (lexical) + dense (embeddings)
- **Périmètre cadré**
événements Pays de la Loire < 1 an
- **Réponses sourcées**
citations des documents récupérés

Ses limites = nos 4 défis

- 1 **Stateless**
aucune mémoire de conversation
- 2 **Pas de contexte géographique**
impossible de filtrer par distance
- 3 **Connaissance figée**
pas d'accès web temps réel
- 4 **Aucun monitoring**
ni qualité, ni coût, ni satisfaction

Quatre défis + des contraintes transverses

1

Mémoire conversationnelle

Historique de session + personnalisation persistante

2

Contexte géographique

Recherche par rayon / distance (« à 20 km de moi »)

3

Recherche web temps réel

Compléter le RAG sur l'actualité la plus fraîche

4

Monitoring

Qualité du RAG, coût par requête, satisfaction

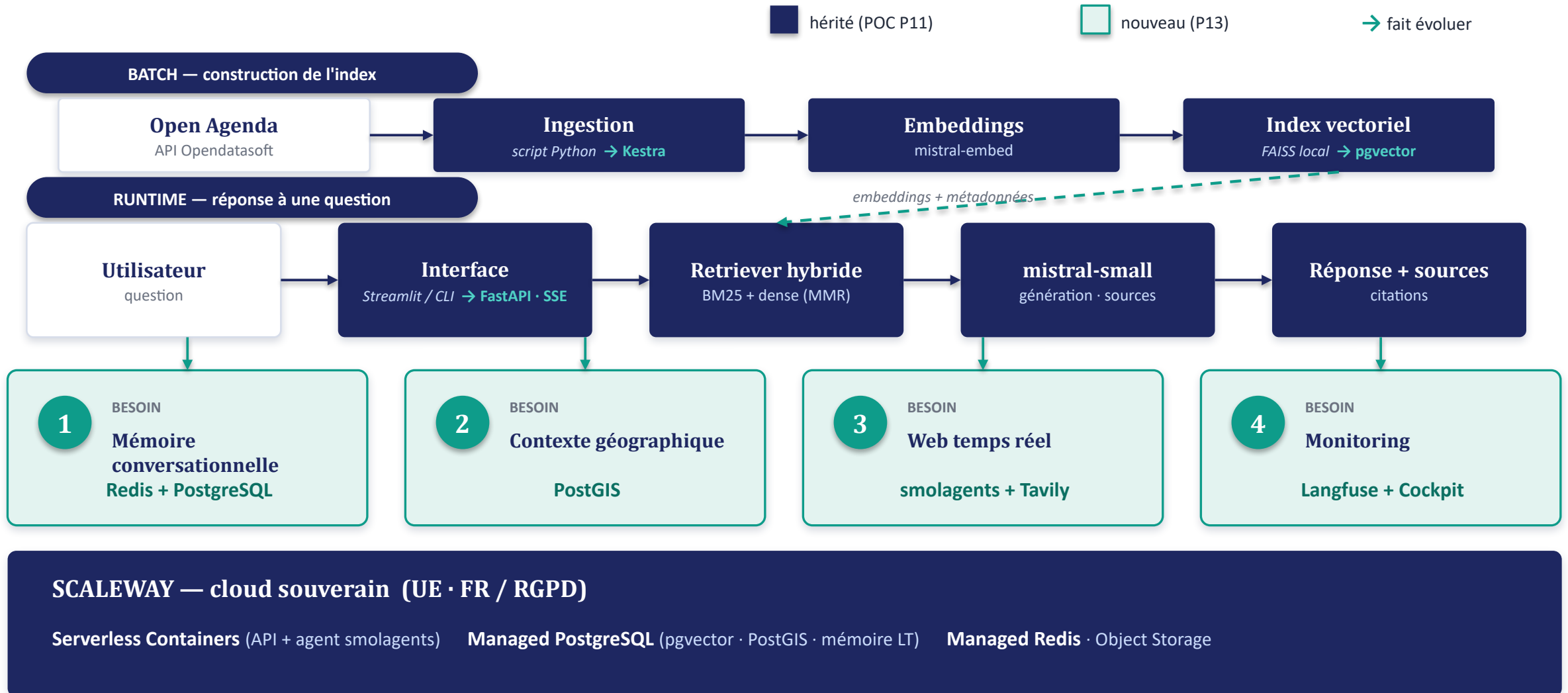
Contraintes transverses

Souveraineté FR / RGPD

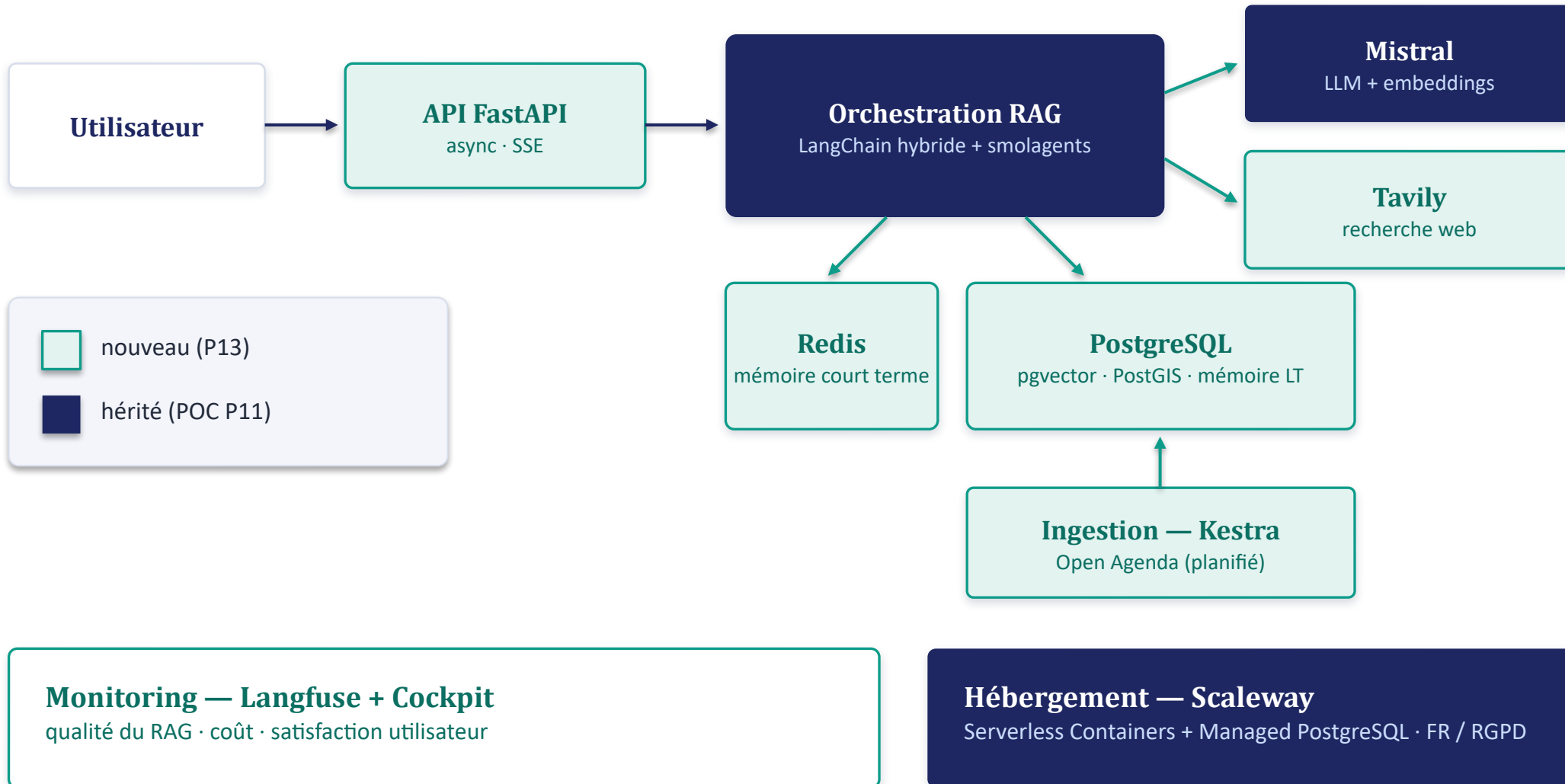
Scalabilité multi-instances

Budget MVP maîtrisé

Le schéma du POC P11, augmenté par les besoins



Architecture cible du MVP



Un choix par défi — besoin, option, critère

1

Mémoire : court terme rapide + long terme durable

Redis + PostgreSQL

Critère décisif — Deux besoins de natures différentes : volatil/rapide vs durable/requêtable

Vigilance : cohérence des deux stores (TTL vs persistance)

2

Géo : requêtes rayon / distance natives

PostGIS

Critère décisif — Pousser le calcul géo dans la base, pas dans l'application

Vigilance : indexation spatiale GIST à prévoir

3

Web : compléter le RAG sur l'actualité

smolagents + Tavily

Critère décisif — Orchestration agentique légère + search pensé pour LLM (piste du brief)

Vigilance : coût par requête et latence ajoutée

4

Monitoring : qualité, coût, satisfaction

Langfuse + Cockpit

Critère décisif — Observabilité LLM-native (traces, coût, eval) + souveraineté/gratuité

Vigilance : définir les métriques tracées dès le départ

Arbitrages structurants & socle hérité

VECTOR STORE

écarté : FAISS · Qdrant

pgvector

Unifie vecteurs + géo + relationnel dans une seule base, moins d'ops pour un MVP.

Qdrant = plan B si la charge explose.

CLOUD

écarté : GCP · AWS · OVHcloud

Scaleway

Souveraineté FR/RGPD, juridiction UE, cohérent avec Mistral (FR).

Pas le moins cher : critère = souveraineté, assumé.

LLM + EMBEDDINGS

écarté : OpenAI · self-hosted

Mistral

Hérité et validé en P11, FR/RGPD, abstrait via LangChain.

Remplaçable sans réécrire le RAG.

Socle hérité — sans arbitrage

FastAPI (async + SSE) **LangChain** (RAG hybride réutilisé) **Kestra** (ingestion, maîtrisé P10/P12)

Quelles métriques, et où les mesurer

Qualité du RAG

Faithfulness

LLM-juge, échantillonné

Answer relevancy

pertinence réponse

Recall@k / MRR

offline, déjà outillé

Taux de « je ne sais pas »

comptage simple

Coût

€ par requête

le chiffre clé

Tokens in / out

décompose le coût

Coût / jour & / user

projection à l'échelle

LLM / embed / web

répartition

Satisfaction

Feedback + / -

explicite (pouce)

Clic sur sources

confiance

Reformulations

signal implicite

Abandon / durée

engagement

Technique

Time-to-first-token

latence perçue (SSE)

Latence p50 / p95

la queue = pannes

Taux d'erreur

5xx, timeouts LLM

Infra (Cockpit)

CPU/RAM, Redis, PG

Langfuse → qualité · coût · feedback (LLM / RAG)

Cockpit → infra Scaleway (conteneurs, ressources)

Starter MVP — 6 métriques à instrumenter dès la Phase 3

€ / requête

Latence p95

Taux d'erreur

Faithfulness

Feedback + / -

Recall@k

Délais & budget

Phasage — estimation ~10 semaines

0	Phase 0 — Socle & infra Docker, Scaleway, Postgres (pgvector + PostGIS)	~2 sem
1	Phase 1 — Mémoire + Géo Redis + Postgres LT, requêtes PostGIS	~3 sem
2	Phase 2 — Web + API smolagents + Tavily, API FastAPI SSE	~3 sem
3	Phase 3 — Monitoring Langfuse, métriques, durcissement	~2 sem

Budget — estimation ~ 160 € / mois

Hyp. : 1 000 req/j · ~4k tokens/req · 20 % via web · Postgres non-HA

Mistral — génération mistral-small · 105M in + 15M out	~14 €
Mistral — embeddings requêtes + ré-indexation	~2 €
Tavily — recherche web ~6 000 recherches / mois	~40 €
Scaleway — PostgreSQL managé, prod non-HA (PRO2-XXS)	~80 €
Scaleway — Serverless + Redis conteneurs à l'usage	~25 €
Langfuse + Cockpit self-host / natif Scaleway	gratuit
Total estimé	~160 €

Dominé par l'infra fixe à ce volume — le LLM ne pèse que ~10 % mais scale avec le trafic. HA Postgres double la base. Grilles juin 2026, à valider.

SYNTHÈSE

Un MVP souverain, justifié défi par défi

- **4 défis couverts** — une techno par défi, chaque choix tracé jusqu'à un besoin
- **Stack souveraine** — Scaleway + Mistral, FR / RGPD de bout en bout
- **pgvector unifie** — vecteurs + géo + relationnel dans une seule base pour le MVP
- **Socle réutilisé** — LangChain + RAG hybride du POC P11, on ne réinvente pas

Prochaine étape → valider les hypothèses de budget, puis lancer la Phase 0 (socle & infra)